Habib University
shaping futures

CS201 – Data Structures II

Lecture 05

Dr. Muhammad Mobeen Movania

Overview

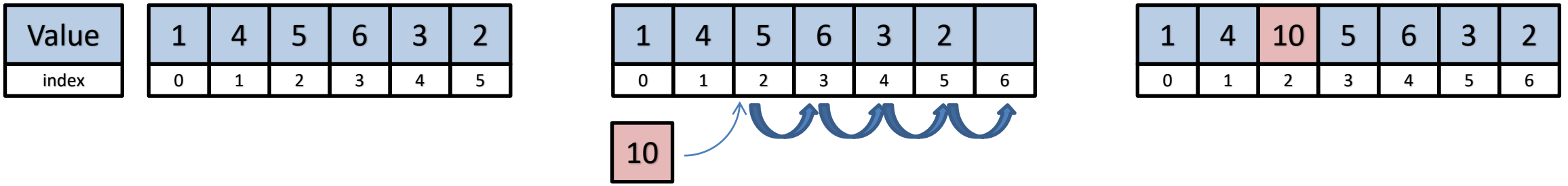
- Issues with Array based Lists
- Introduction to LinkedLists
- SLList (Singly LinkedLists)
  - Stack operations (push/pop)
  - Queue operations (enqueue/dequeue)
- DLList (Doubly LinkedLists)
  - Add/remove operations
- Summary

Issues with Array-based lists(1)

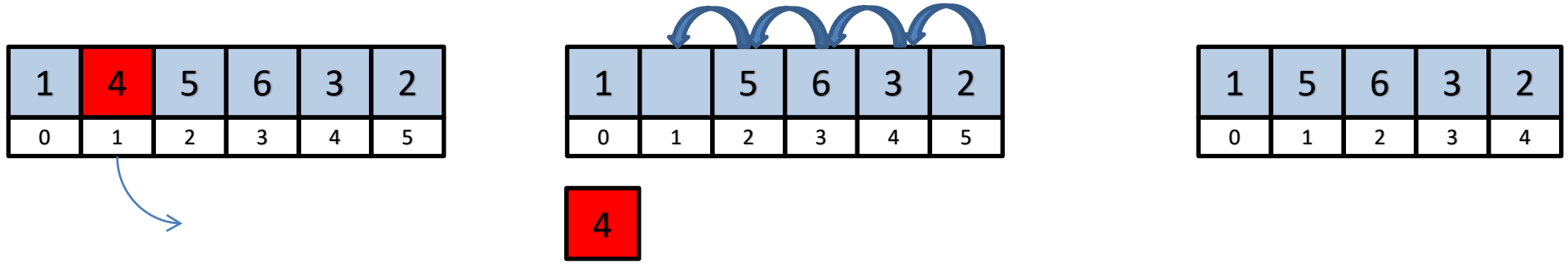
- Arrays
  - Collection of a number of values of a similar data type
  - Requires contiguous memory locations
  - Access is O(1), any value can be accessed directly through an indexer (array[i])
  - Searching is O(N) for an array of N elements
- Insertion and Deletion of a value at a location i are cumbersome
  - Insertion: You have to create a new array of size (N+1), copy values by movement of all items at location >i by one place to the right
  - Deletion requires movement of all items at location >i by one place to the left
- Large array allocations are impossible if memory is already scarce

Issues with Arrays (2)

- Insertion of value 10 at index 2
  - Move all values from index 2 to 5 one place to the right



- Deletion of value 4 from index 1
  - Move all values from index 2 to 5 one place to the left

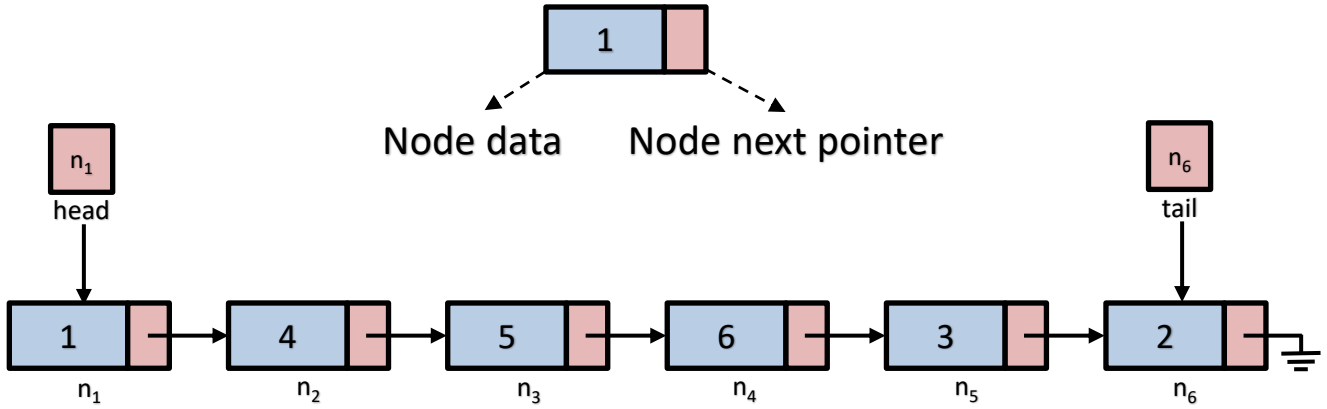


Introduction to LinkedList

- Create nodes dynamically and link them through pointers
- No limit on the number of nodes creatable (memory is the only limit)
- Advantages
  - Insertion and Deletion are easy, you just need to update a few pointers
- Disadvantages
  - We cannot access a node using index, instead we have to walk from one node to another until we find the node we want.

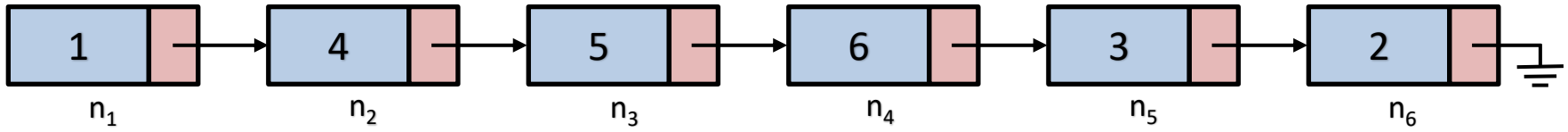
Basic Structure of LinkedList

- Assuming the data array=[1,4,5,6,3,2]

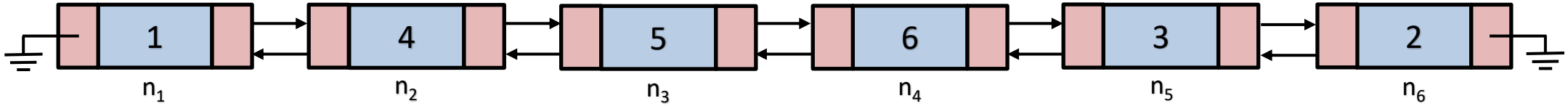


Types of LinkedList

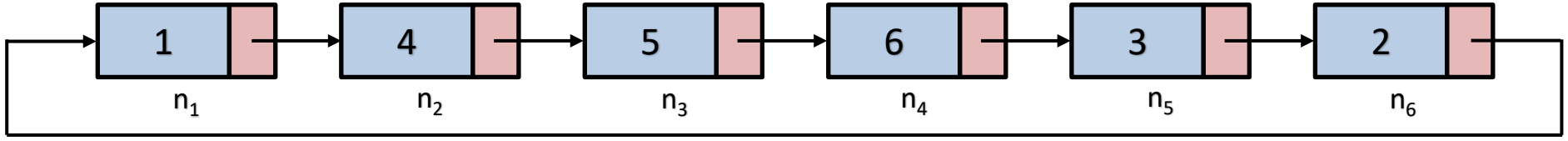
Singly LinkedList (SLList)



Doubly LinkedList (DLList)

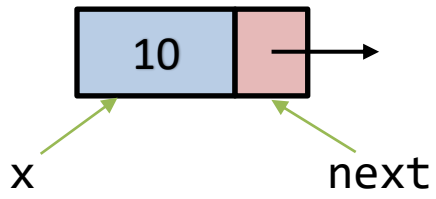


Circular LinkedList



Basic Node Structure (SLList)

```
template<typename T>
struct Node
{
    T x;
    Node* next;
    Node(T x0)
    {
       x = x0;
       next = nullptr;
    }
};
```



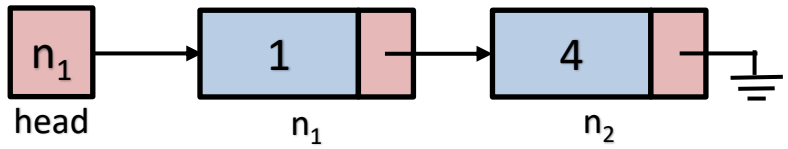

```
template<typename T>
class SLList
{
    Node<T>* head;
    Node<T>* tail;
    int n;
    //rest of the implementation
};
```

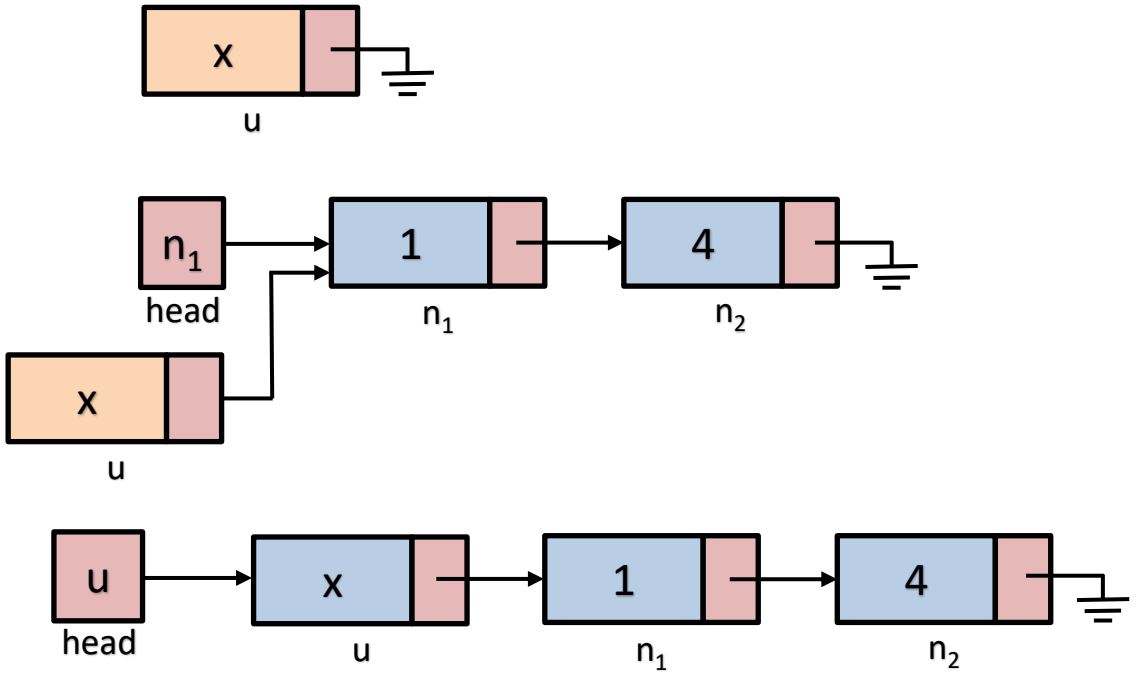
Implementing stack push operation using SLList

- Push and pop functions can be easily implemented in SLList
- Push operation needs 4 steps
  - Allocate new SLList node called u
  - Set u.next to the current SLList head node
  - Make u as the new head node of SLList
  - Increment n by 1
- Complexity
  - O(1)

```
T push(T x) {
 Node *u = new Node(x);
 u->next = head;
 head = u;
 if (n == 0)
    tail = u;
 n++;
 return x;
}
```

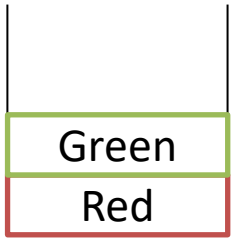
Visualization of add(push) operation

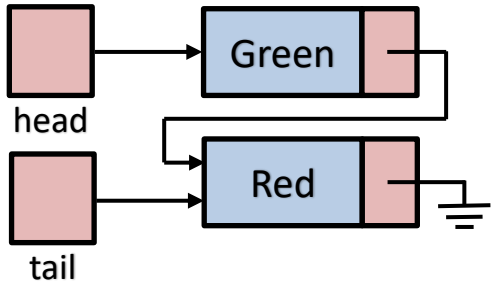




Example – Stack push operations

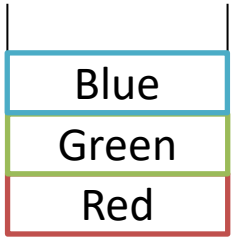
- Current stack state

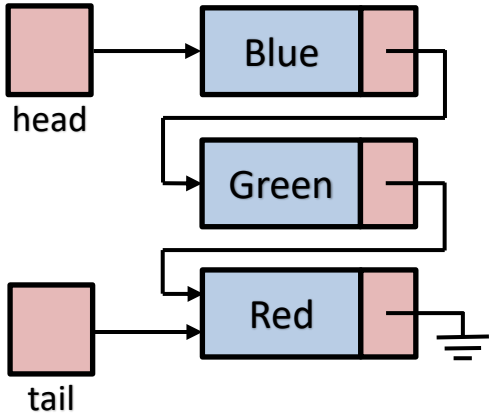




n=2

- After stack.push(“Blue)





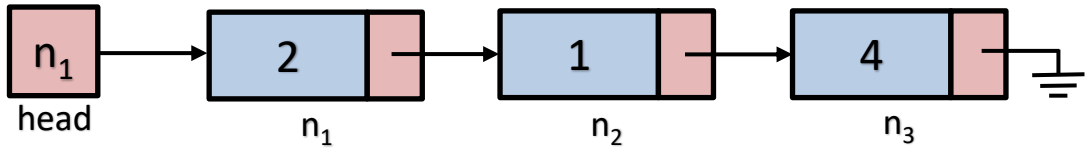
n=3

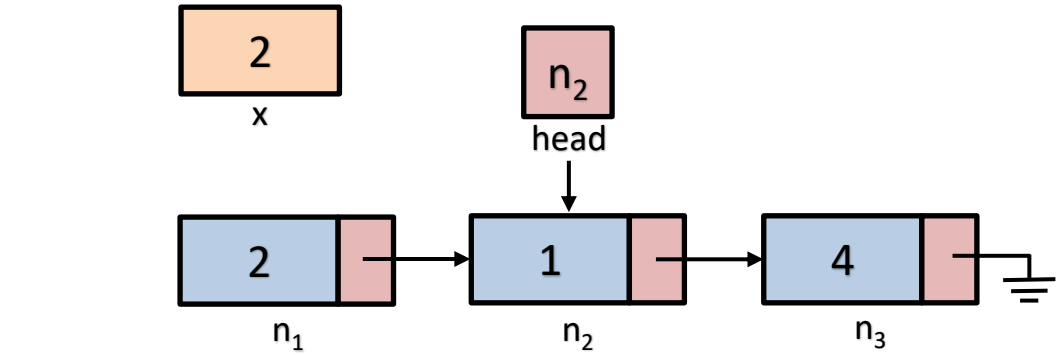
Implementing stack pop operation using SLList

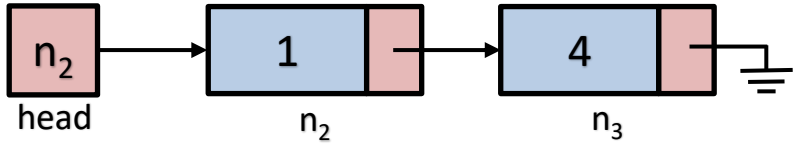
- Pop operation needs 6 steps
  - Store the current head node's value
  - Store a reference to the current head node
  - Set head to the next node
  - Delete the head using the reference from step 2
  - Decrement n by 1
  - Return the old head's value stored in step 1
- Complexity
  - O(1)

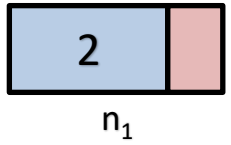
```
T pop() {
    if (n == 0)
        return null;
    T x = head->x;
    Node *u = head;
    head = head->next;
    delete u;
    if (--n == 0)
        tail = nullptr;
    return x;
}
```

Visualization of remove(pop) operation









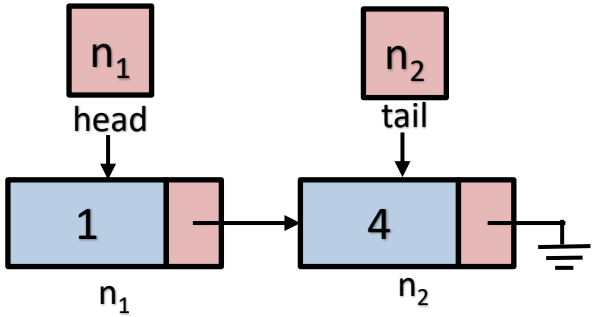
Implementing queue add(enqueue) operation using SLList

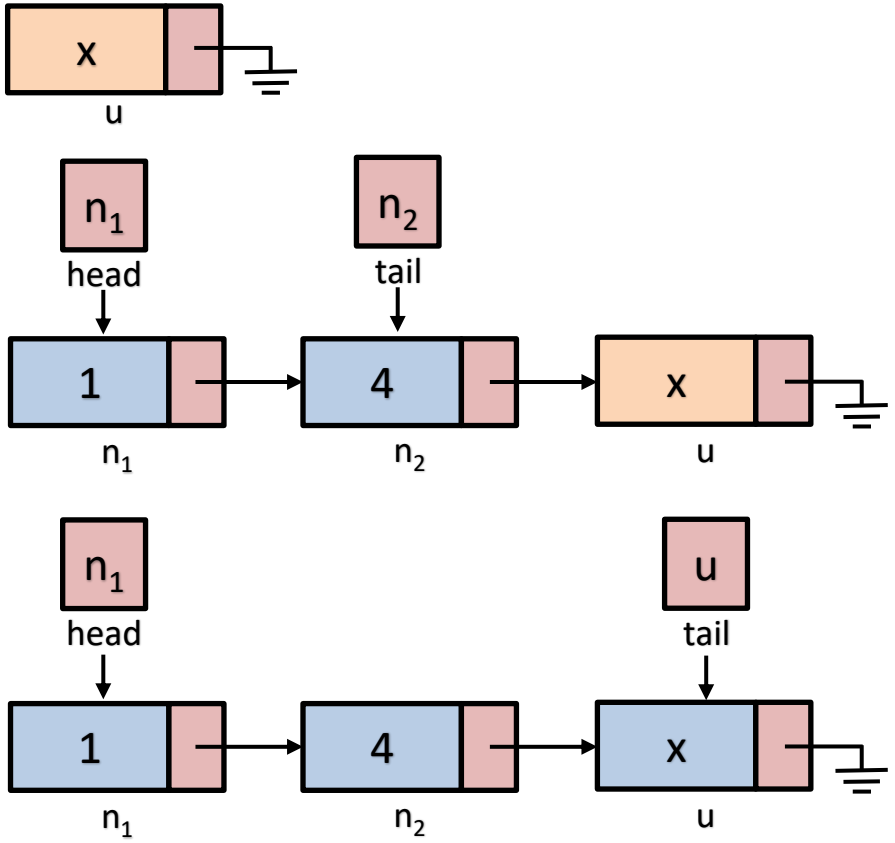
- Addition (enqueue) takes place at the tail end in 5 steps
  - Allocate a new node u
  - if the queue is empty
    - Set u as the head node
  - Otherwise
    - Set u to tail's next pointer
  - Set u as the tail node
  - Increment n

- Complexity
  - O(1)

```
bool add(T x) {
 Node *u = new Node(x);
 if(n==0)
    head = u;
 else
    tail->next = u;
 tail = u;
 n++;
 return true;
}
```

Visualization of add(enqueue)



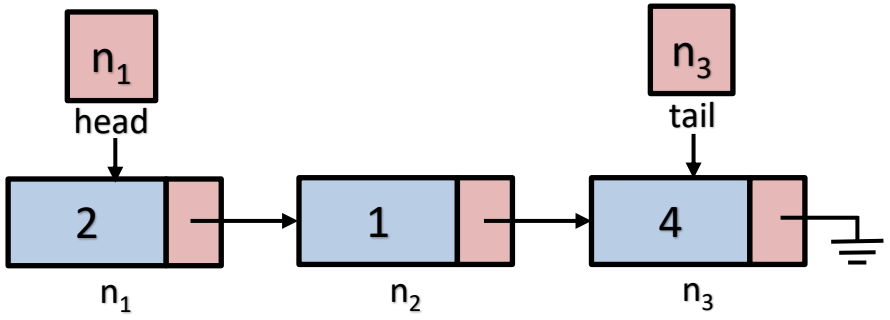


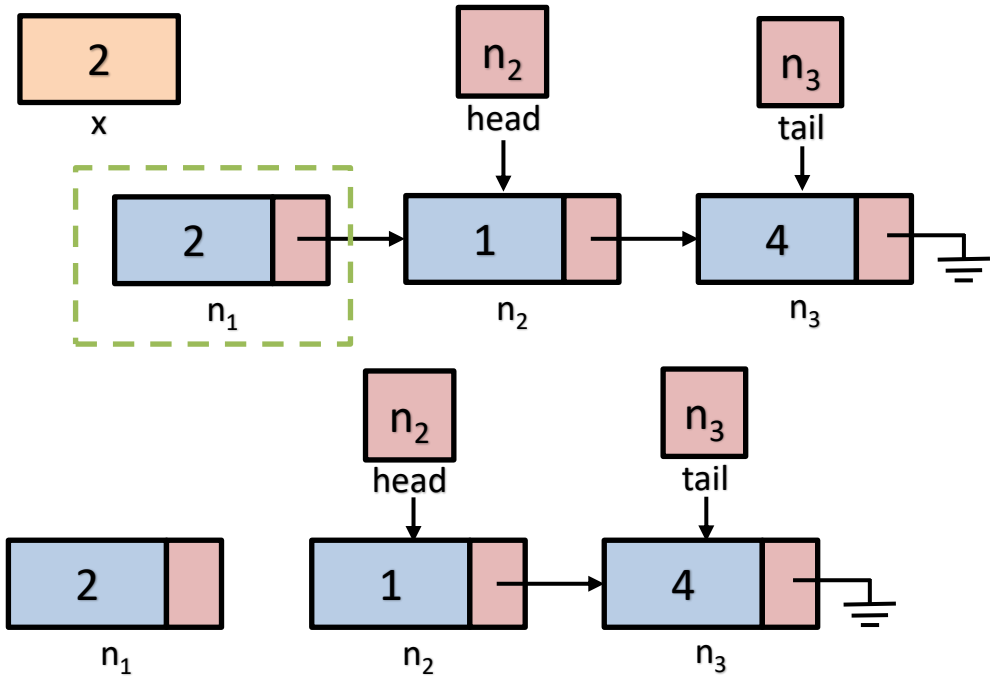
Implementing queue remove(dequeue) operation using SLList

- Remove (dequeue) takes place at the head end in 6 steps
  - Store the current head node's value
  - Store a reference to the current head node
  - Set head to the next node
  - Delete the head using the reference from step 2
  - Decrement n by 1
  - Return the old head's value stored in step 1
- Complexity
  - O(1)

```
T remove() {
    if (n == 0)
        return null;
    T x = head->x;
    Node *u = head;
    head = head->next;
    delete u;
    if (--n == 0)
        tail = NULL;
    return x;
}
```

Visualization of remove (dequeue)



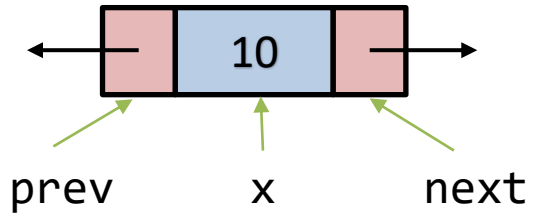


Quick question?

- What will be the complexity of removing a node from the tail of the SLList?
  - O(1)
  - O(n)
  - $O(n^2)$
  - $O(\log_2 n)$

Basic Node Structure (DLList)

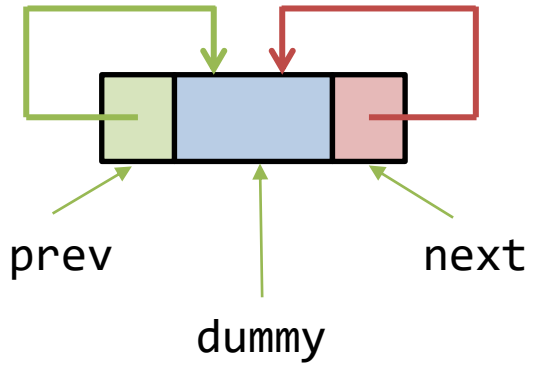
```
template<typename T>
struct Node
{
    T x;
    Node* next;
    Node* prev;
    Node(T x0)
    {
       x = x0;
       next = nullptr;
       prev = nullptr;
    }
}
```



DLList structure

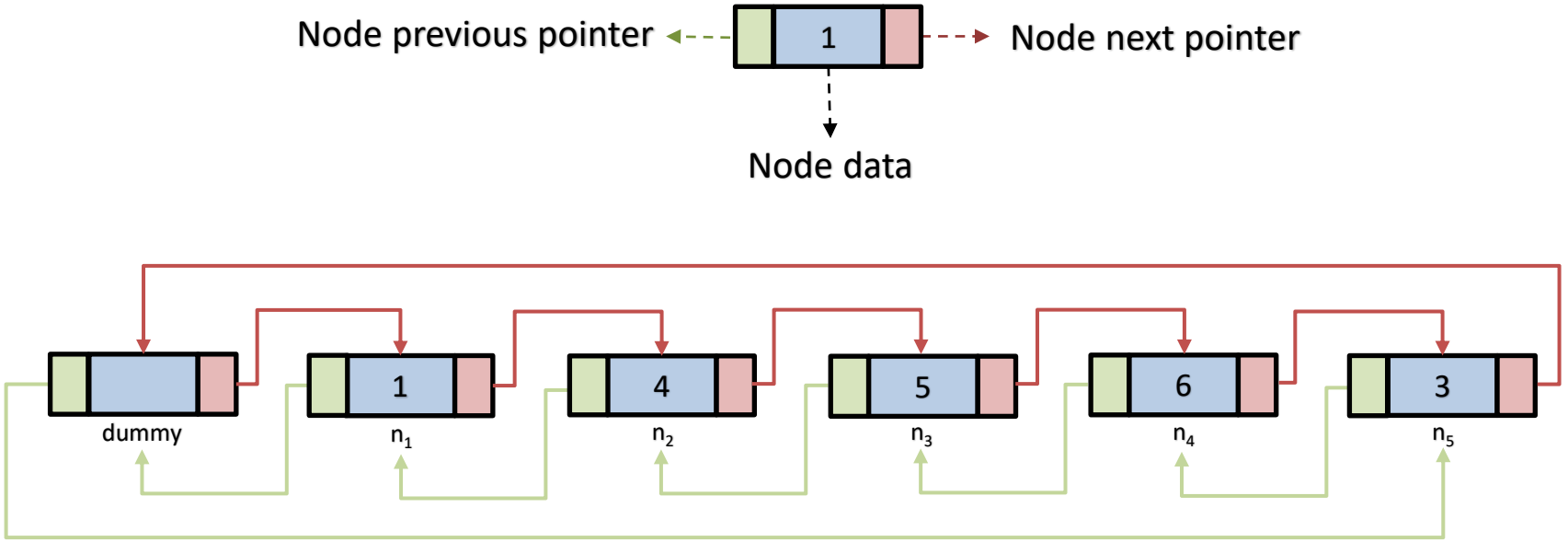
- We do not store a head or tail pointer in a DLList instead we store a dummy node
  - You don't need head or tail pointer
  - It helps connect the last node of the DLList to the first node

```
template<typename T>
class DLList
{
    Node<T>* dummy;
    int n;
    public:
      DLList()
      {
        dummy.next = &dummy;
        dummy.prev = &dummy;
        n = 0;
      }
}
```



Example visualization of a DLList

- Assuming the data array=[1,4,5,6,3]



DLList::getNode - finding a node at a particular index i

- We determine based on the given index i
  - If i<n/2
    - We work forward using the dummy->next
  - Otherwise
    - we work backward using the dummy->prev
- Complexity
  - O(1+min(i, n-i))

```
Node<T>* DLList::getNode(int i) {
    Node<T>* p;
    if (i < n / 2) {
        p = dummy.next;
        for (int j=0; j<i; j++)
            p = p->next;
    } else {
        p = &dummy;
        for (int j=n; j>i; j--)
            p = p->prev;
    }
    return (p);
}
```

DLList accessor and mutator

- Both get and set functions call the getNode function to get the node and then return or set its value

```
T DLList::get(int i)
{
    return getNode(i)->x;
}
```

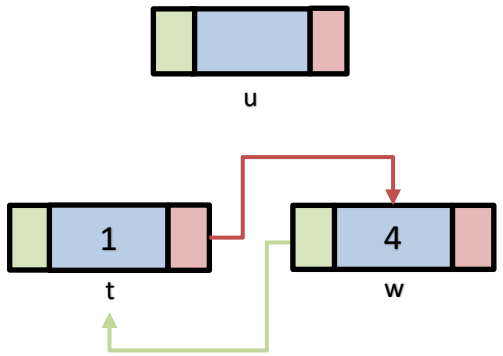
```
T DLList::set(int I, T x)
{
    Node<T>* p = getNode(i);
    T y = p->x;
    p->x = x;
    return y;
}
```

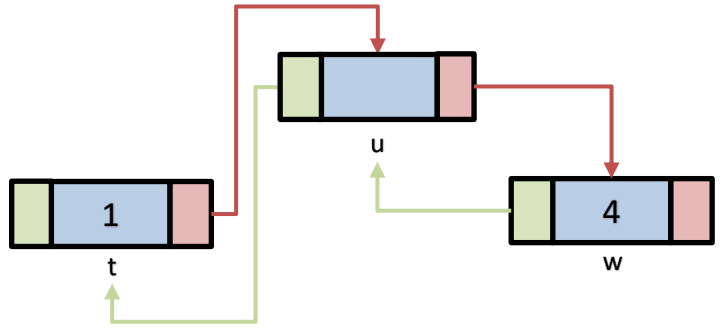
- Complexity
  - O(i+min(i,n-i)) - the cost of getNode

DLList::add/addBefore functions

```
Node<T>* DLList::addBefore(Node<T> *w, T x)
{
    Node<T> *u = new Node<T>;
    u->x = x;
    u->prev = w->prev;
    u->next = w;
    u->next->prev = u;
    u->prev->next = u;
    n++;
    return u;
}
```

```
void DLList::add(int i, T x)
{
    addBefore(getNode(i), x)
}
```





- Complexity
  - O(i+min(i,n-i)) - the cost of getNode

DLList::remove function

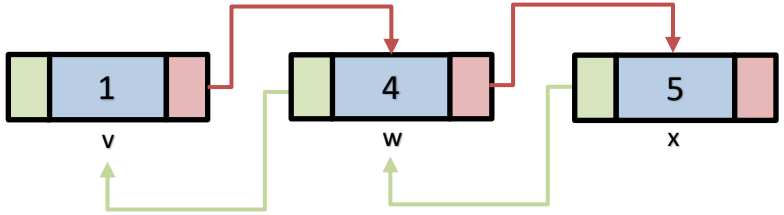
- To remove the node w, update w's next and prev pointers so that they skip over w

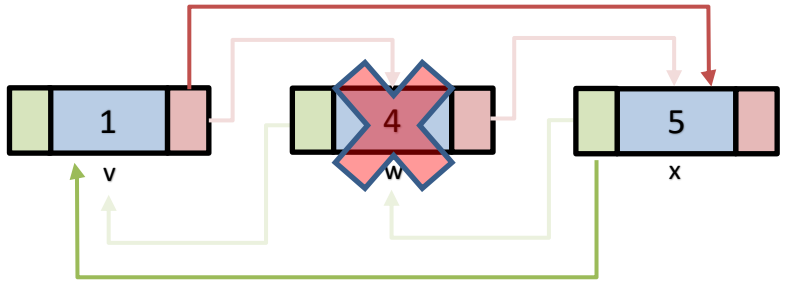
```
void DLList::remove(Node<T> *w)
{
   w->prev->next = w->next;
   w->next->prev = w->prev;
   delete w;
   n--;
}
```

```
T DLList::remove(int i)
{
   Node<T> *w = getNode(i);
   T x = w->x;
   remove(w);
   return x;
}
```

- Complexity
  - O(i+min(i,n-i))

Visualization of DLList::remove operation





Complexity of finding the node – DLList::getNode

- We see that in most operations in DLList the cost is dominated by the getNode function which is O(i+min(i,n-i))

- This cost may be lowered to constant cost O(1) by storing the references to nodes in a USet implemented as a ChainedHashTable

Complexities – Operations using SLList

Data Structure::Operation	Complexity
Stack::push	O(1)
Stack::pop	O(1)
Queue::enqueue	O(1)
Queue::dequeue	O(1)
Deque::addFirst	O(1)
Deque::addLast	O(1) (if have tail pointer) O(n) otherwise
Deque::removeFirst	O(1)
Deque::removeLast	O(1) (if have tail pointer) O(n) otherwise
getNode	O(n)

Complexities – Operations using DLList

Data Structure::Operation	Complexity
Stack::push	O(1)
Stack::pop	O(1)
Queue::enqueue	O(1)
Queue::dequeue	O(1)
Deque::addFirst	O(1)
Deque::addLast	O(1)
Deque::removeFirst	O(1)
Deque::removeLast	O(1)
getNode	O(1+min(i, n-i))

Summary

- We went through an introduction to LinkedLists
- We saw the different variants of linked list including SLList and DLList
- We saw how we may implement stack and queue operations in the SLList
- We saw how to implement add and remove operations in a DLList

Book Readings

- ODS: Chapter 3 pages: 63-71

Next Lecture

- Lecture 06 – Skiplists